

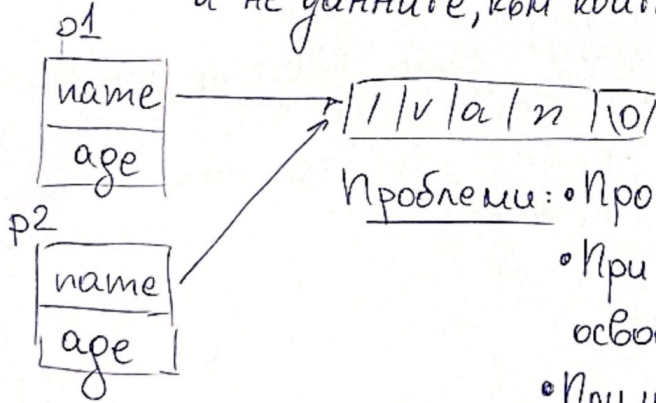
Копиращ конструктор. Оператор =. Голяма четворка в C++.

Семинар 5
Серхан

Голяма четворка - 4 специални функции, които всеки клас притежава

- Конструктор по подразбиране - при създаване на обект без аргументи
- Копиращ конструктор - създаване на обект като копие на съществуващ
- Оператор = - присвояване на стойност на вече съществуващ обект
- Деструктор - унищожаване на обект
- При клас без динамична памет - компилаторът генерира четворката автоматично

Shallow copy - при динамична памет и без реализация на четворката се копира стойността на указателя, а не данните, към които сочи.



- Проблеми:
- Промяна на p2 - променя и p1.
 - При унищожаване на p2 паметта се освобождава
 - При унищожаване на p1 се вика delete[] върху освободена памет

Deep copy - при динамична памет; зарежа нова памет и копира съдържанието. 2та обекта имат независими копия на данните.

- Промяна или унищожение на единия обект не засяга другия.
- При клас с динамична памет - ръчно имплементираме Голямата Четворка.

1. Конструктор по подразбиране

- поставя обекта във валидно начално състояние
- указателите се инициализират с null ptr.
- компилаторът не го генерира, ако сме дефинирали друг конструктор → трябва да го дефинираме ние

2. Копиращ конструктор

- при създаване на нов обект като копие на съществуващ.

```
Person (const Person & other)
```

```
{ copyFrom(other);
```

```
}
```

3. Оператор =

- когато обект вече съществува и му се присвоява нова стойност

```
Person & operator = (const Person & other)
```

```
{ if (this != &other) → проверяваме дали other не сочи към  
                        същия обект като this
```

```
{ free();
```

```
  copyFrom(other); → записваме старата памет
```

```
}
```

```
return *this;
```

```
}
```

4. Деструктор

- освобождава всички ресурси, притежавани от обекта

```
~Person()
```

```
{ free();
```

```
}
```

- при деструкторите реда на извикване е обратен спрямо конструкторите

*Помощни функции

```
void free()  
{ delete[] name;  
  name = nullptr;  
}
```

```
void copyFrom(const Person& other) {  
  name = new char[strlen(other.name)+1];  
  strcpy(name, other.name);  
  age = other.age;  
}
```

*Скрито копиране при подаване по стойност

```
void printPerson(Person p) ← копиране  
{ p.print();  
}
```

→ деструктор

по-добре и безопасно:

```
void printPerson(const Person& p) ← без копиране  
{ p.print();  
}
```